GUÍA COMPLETA DE COMANDOS GIT

Opciones, símbolos, parámetros y ejemplos

Referencias a las páginas originales y errores señalados

MATERIAL DE CONSULTA PARA LA UNIDAD

Guía de lectura. Cada entrada incluye el comando, su función, el significado de sus partes y las páginas del *PDF original* donde aparece. Los comandos presentes solo en esquemas se identifican expresamente.

> **Convenciones:** las páginas que se citan son las del archivo original de 30 páginas, no las de esta guía. El texto escrito entre < > representa un valor que hay que sustituir. Las correcciones de errores del temario se reúnen al final.

Abarca · Todas las formas y ejemplos de comandos incluidos en el documento: Git y las órdenes externas mencionadas.

También explica · Opciones cortas y largas, nombres de ramas, parámetros, comillas y símbolos como *, ., .., =, # y &&.

Para estudiar · El significado de cada elemento del comando es tan importante como recordar la orden completa.

01 Cómo se lee un comando

En Git, una instrucción suele tener un **programa**, un **subcomando**, opciones (que modifican el comportamiento) y argumentos (que aportan datos). No es lo mismo escribir una opción de una letra que una de palabra completa.

```
git commit -am "Corrijo la práctica"
```

Elemento	Significado en este ejemplo
`git`	Programa que interpreta la orden.
`commit`	Operación: crear una confirmación de cambios.
`-a`	Opción corta: incorporar al commit los cambios en archivos con seguimiento.
`-m`	Opción corta: utilizar el argumento siguiente como mensaje del commit.
`-am`	Dos opciones cortas agrupadas: equivale a escribir **-a -m**.
`"Corrijo la práctica"`	Mensaje de confirmación; se conserva como un solo argumento gracias a las comillas.

Muy importante: `-p` en `git add -p` no significa lo mismo que en `git log -p`. Las opciones se interpretan siempre en el contexto del comando al que pertenecen.

Las cuatro distinciones que más se confunden

Opción corta / larga. `-a` y `-m` utilizan **un** guion; `--amend` y `--force` utilizan **dos**.

Dos opciones / una palabra. `-a -m` y `-am` tienen igual efecto en commit, pero `--amend` es una opción diferente, no es una combinación.

Punto / asterisco. `git add .` usa una ruta de directorio. `git add *` usa un comodín, habitualmente expandido por el intérprete de comandos.

Commit / push. `git commit` guarda en el repositorio **local**; `git push` envía al remoto los commits correspondientes.

02 Diccionario completo de opciones

El mismo modificador puede tener efectos distintos según la orden. Por eso se indica el **comando donde aparece**, su significado y la página del documento original.

Un solo guion (opciones cortas)

Opción	Se utiliza con	Qué hace	PDF
`-a`	`git commit`	Prepara antes del commit los cambios de archivos con seguimiento. No incorpora archivos nuevos sin seguimiento.	22-23
`-m`	`git commit`	Indica el mensaje de la confirmación; necesita un texto a continuación.	9, 23
`-b`	`git checkout`	Crea una nueva rama cuando se usa checkout -b y cambia a ella.	24
`-s`	`git status`	Muestra el estado con formato abreviado (short).	16
`-b`	`git status`	Incluye información de la rama (branch) en la salida abreviada.	16
`-n`	`git log`	Limita el número de commits mostrados; lleva número como argumento.	17
`-p`	`git add`	Preparación interactiva por fragmentos (patch).	22
`-p`	`git log`	Muestra el parche/diferencias de cada commit; no abre interacción.	17

Combinadas (dos letras juntas)

Opción	Se utiliza con	Qué hace	PDF
`-am`	`git commit`	Equivale, en este uso, a -a -m: se activan ambas opciones y -m consume el siguiente mensaje.	23

Dos guiones (opciones largas)

Opción	Se utiliza con	Qué hace	PDF
`--version`	`git`	Muestra la versión de Git instalada.	8
`--global`	`git config`	Establece configuración del usuario, no solo del repositorio en curso.	9
`--amend`	`git commit`	Sustituye la última confirmación con una nueva versión de ella.	23
`--oneline`	`git log`	Formato compacto: un commit por línea.	14-18
`--decorate`	`git log`	Añade nombres de ramas y etiquetas a los commits.	14-18
`--graph`	`git log`	Dibuja un árbol/gráfico de confirmaciones con caracteres de texto.	15-18
`--all`	`git log`	Consulta el historial desde todas las referencias disponibles.	15-16
`--stat`	`git log`	Incluye estadísticas de archivos y líneas modificadas.	17
`--author=`	`git log`	Filtra commits según el autor indicado tras el signo igual.	17
`--grep=`	`git log`	Filtra por el contenido de los mensajes de commit.	17
`--all`	`git push`	Publica todas las ramas locales en un remoto. Mismo texto, significado ajustado al comando.	19
`--force`	`git push`	Fuerza una actualización que, de otro modo, Git podría rechazar. Puede sobrescribir trabajo remoto.	19
`--no-verify`	`git push`	Omite el hook pre-push del cliente; no evita comprobaciones del servidor.	19
`--hard`	`git reset`	Modo de restablecimiento que descarta cambios de archivos controlados.	24

Solo mencionada en el texto

Opción	Se utiliza con	Qué hace	PDF
`--verify`	`git push`	Forma afirmativa del comportamiento de verificación previo al push, citada en la explicación.	19
`--quiet`	`git pull`	Reduce los mensajes informativos del comando. No aparece desarrollada en un ejemplo completo.	20
`--verbose`	`git pull`	Aumenta el detalle de la salida. No aparece desarrollada en un ejemplo completo.	20

03 Diccionario de símbolos, signos y nombres

Estos caracteres no son detalles decorativos: pueden modificar por completo la interpretación de una instrucción o formar parte de sus argumentos.

Símbolo / término	Cómo se interpreta	Aplicación práctica
`-`	**Un guion delante de una letra**	Introduce normalmente una opción corta: -m, -a, -p, -n.
`--`	**Dos guiones delante de una palabra**	Introduce una opción larga: --global, --amend, --oneline.
`-- (solo)`	**Separador entre opciones y rutas**	Es un uso adicional válido de Git, pero NO aparece como instrucción independiente en la unidad; ejemplo aclaratorio: git log -- README.md.
`*`	**Asterisco como comodín**	En git add * la terminal puede expandir nombres de archivos y carpetas; normalmente no incluye nombres ocultos.
`.`	**Punto separado como argumento**	En git add . indica «esta carpeta y sus subcarpetas»; al final de git clone <URL> . indica «clona aquí».
`..`	**Dos puntos consecutivos sin espacio**	En git log A..B define un rango de commits accesibles desde B y no desde A.
`.git`	**Nombre de directorio / sufijo de URL**	El directorio oculto .git contiene metadatos de Git; una URL puede terminar en .git sin representar el mismo concepto.
`"..." y '...'`	**Comillas dobles y simples**	Agrupan palabras en un único argumento: nombre, correo o mensaje de commit; las comillas delimitadoras no son parte del mensaje.

Símbolo / término	Cómo se interpreta	Aplicación práctica
`<...>`	**Marcadores de posición**	No se escriben literalmente al ejecutar: sustituir <archivo>, <rama>, <URL>, <limite>, etc. por valores reales.
`=`	**Signo de asignación en una opción**	En --author="..." o --grep="...", une la opción larga con el valor o patrón.
`&&`	**Encadenamiento de terminal**	En clone ... && cd ... ordena ejecutar el cambio de carpeta solo si el clonado terminó correctamente.
`#`	**Comentario de terminal**	En los ejemplos de la página 24, el texto después de # es explicación y no se ejecuta; en Bash el símbolo comienza un comentario.
`+`	**Suma explicativa, no terminal**	git fetch + git merge es una explicación conceptual de pull, no una instrucción que deba copiarse tal cual.
`HEAD`	**Referencia a la posición actual**	Indica normalmente el commit de la rama activa; aparece en el log y en los marcadores de conflicto.
`origin`	**Nombre de remoto**	Alias convencional de un repositorio remoto; no es sinónimo de GitHub y se puede configurar con otra URL.
`main / master / testing`	**Nombres de ramas**	Son nombres concretos de ramas, no palabras reservadas: usar el que exista en cada repositorio.
`NOMBRE-REMOTO / RAMA-REMOTA`	**Argumentos explicativos**	Marcan valores que hay que reemplazar, igual que REPOSITORIO, REFSPEC u OPCIONES.
`y, n, s, e, q`	**Respuestas a git add -p**	y añade un fragmento; n lo omite; s lo divide; e lo edita; q termina.
`Espacio / q`	**Teclas del visor de git log**	Espacio avanza en la paginación y q abandona el visor cuando se usa un paginador. No son opciones de git log.
`<<<<<<< / ======= / >>>>>>>`	**Marcadores de conflicto**	Delimitan alternativas durante un conflicto de fusión; NO son comandos para teclear. Se muestran en la página 25.

Ejemplo externo, solo para aclarar el separador `--`: `git log -- README.md` expresa que *README.md* es una ruta de archivo y no una rama o una opción. Esta variante es una explicación técnica añadida: **no está escrita literalmente en la unidad**.

04 Catálogo completo de instrucciones

En cada ficha se explica **lo que hace** y se desglosan **todas las opciones y parámetros relevantes**. Cuando una frase del apunte es incorrecta, aquí se muestra el funcionamiento real y después se identifica la errata en el apartado 06.

01 Instalación y configuración

`git --version` P. 8

Consulta la versión de Git instalada y permite comprobar que el programa está disponible.

Partes: git = programa; --version = opción larga que solicita la versión.

`brew install git` P. 9

Instala Git en macOS a través del gestor de paquetes Homebrew.

Partes: brew = gestor de paquetes; install = instalar; git = paquete.

`git config --global user.name "Tu nombre"` P. 9

Define el nombre de autor que Git asociará a las confirmaciones de este usuario.

Partes: config = configuración; --global = se aplica al usuario; user.name = propiedad; comillas = agrupan el nombre.

`git config --global user.email "tu_email@example.com"` P. 9

Define el correo que figurará como autor de las confirmaciones.

Partes: user.email = propiedad del correo; el texto entre comillas debe sustituirse por el correo real.

02 Crear, conectar y clonar repositorios

`git init` P. 9, 11

Inicializa un repositorio en la carpeta actual y crea su directorio .git.

Partes: init = inicializar. No sube nada a Internet y no necesita obligatoriamente una carpeta vacía.

`git remote add origin <URL>` P. 11 · esquema

Registra un repositorio remoto con el alias origin. Figura en el esquema, no en el texto principal.

Partes: remote = remotos; add = añadir; origin = nombre asignado; <URL> = dirección del repositorio.

`git clone https://github.com/usuario/nombre_repo.git` P. 11

Crea una copia local del repositorio remoto en una carpeta nueva llamada como el repositorio.

Partes: clone = clonar; https://... = dirección del origen; el sufijo .git de la URL no es el directorio .git local.

`git clone https://github.com/usuario/nombre_repo.git .` P. 12

Clona el repositorio dentro de la carpeta actual, que debe estar vacía.

Partes: El punto final (.) es el destino: «este directorio»; no forma parte de la URL.

`git clone <URL> && cd <carpeta>` P. 18

Primero clona un repositorio; solo si termina correctamente, entra en su directorio. Aparece como ejemplo combinado.

Partes: <URL> = origen; && = ejecutar lo siguiente solo si lo anterior tuvo éxito; cd = cambiar de carpeta.

03 Preparación de cambios: add

`git add *` P. 9

Prepara las rutas que coincidan con el asterisco expandido por la terminal. Puede dejar fuera elementos ocultos.

Partes: add = preparar; * = comodín de la consola que suele coincidir con nombres no ocultos.

`git add LICENSE` P. 9

Prepara los cambios del archivo concreto llamado LICENSE.

Partes: LICENSE = nombre real de archivo, no opción ni palabra especial.

`git add README test.rb LICENSE` P. 12

Prepara esos tres archivos dentro de una sola instrucción.

Partes: Cada nombre separado por espacio es otro argumento: README, test.rb y LICENSE.

`git add <archivo>` P. 18, 22

Prepara para el siguiente commit los cambios correspondientes a un archivo.

Partes: <archivo> es una indicación genérica: se sustituye por una ruta real, sin escribir < ni >.

`git add <directorio>` P. 22

Prepara los cambios correspondientes al directorio indicado y su contenido.

Partes: <directorio> representa una ruta de carpeta que el usuario debe sustituir.

`git add .` P. 11 · esquema, 24

Prepara los cambios existentes bajo el directorio actual (incluye modificaciones, altas y eliminaciones detectadas en esas rutas).

Partes: . significa carpeta actual. NO es igual a * y no es la carpeta oculta .git.

`git add -p` P. 22

Permite preparar cambios por fragmentos mediante preguntas interactivas.

Partes: -p = patch (fragmentos). Dentro del asistente: y, n, s, e y q tienen acciones diferentes.

04 Confirmar cambios: commit

`git commit` P. 22

Confirma los cambios que ya están preparados; abre el editor para redactar el mensaje.

Partes: commit = crear una confirmación en el historial local. No la publica en GitHub.

`git commit -m "mensaje"` P. 9-10, 12, 23-24

Crea una confirmación y toma el texto proporcionado como mensaje, sin abrir el editor.

Partes: -m = message; "mensaje" = texto, no una opción. Se muestra con distintas frases en las páginas citadas.

`git commit -a` P. 22

Prepara automáticamente los cambios en archivos ya controlados y crea el commit; abre el editor.

Partes: -a = all (archivos con seguimiento: modificados y eliminados). No incluye archivos nuevos sin seguimiento.

`git commit -a -m "mensaje"` P. 15

Confirma cambios de archivos controlados e introduce directamente el mensaje.

Partes: -a = preparar archivos controlados; -m = indicar el mensaje. Son dos opciones escritas por separado.

`git commit -am "mensaje"` P. 23

Tiene el mismo efecto que utilizar -a y -m por separado en este caso.

Partes: -am = combinación de las opciones cortas -a y -m; la m recibe el texto siguiente.

`git commit --amend` P. 23

Reemplaza el último commit por otro que puede incluir cambios preparados o modificar el mensaje.

Partes: --amend = enmendar la última confirmación. No se escribe --amed ni –amend.

05 Inspeccionar el estado: status

`git status` P. 10, 16

Describe la rama actual y los archivos con cambios pendientes o sin seguimiento.

Partes: status = estado; distingue entre lo preparado y lo que todavía está solo en la carpeta de trabajo.

`git status -s` P. 16

Muestra una versión abreviada del estado utilizando indicadores de estado por archivo.

Partes: -s = short (corto). Es una opción corta con un solo guion.

`git status -s -b` P. 16

Muestra el estado abreviado e incluye información de la rama.

Partes: -s = formato corto; -b = branch (información de la rama). Son dos opciones.

06 Consultar el historial: log

`git log` P. 14, 17

Muestra el historial de confirmaciones realizadas en el repositorio.

Partes: log = historial; el visor suele usar Espacio para avanzar y q para salir.

`git log -n <límite>` P. 17

Limita la salida a una cantidad máxima de confirmaciones recientes.

Partes: -n = número máximo; <límite> debe sustituirse por un número, por ejemplo 5.

`git log --oneline` P. 17

Resume cada confirmación en una línea.

Partes: --oneline = formato breve con identificador abreviado y mensaje.

`git log --stat` P. 17

Añade estadísticas de archivos y líneas modificadas a cada confirmación.

Partes: --stat = estadísticas de cambios.

`git log -p` P. 17

Presenta el parche: líneas añadidas y eliminadas en los commits.

Partes: -p = patch (diferencias). NO cumple la misma función que -p en git add.

`git log --author="<patrón>"` P. 17

Filtra las confirmaciones según el autor.

Partes: --author = filtro de autor; = asigna el patrón; las comillas agrupan el texto de búsqueda.

`git log --grep="<patrón>"` P. 17

Filtra las confirmaciones según coincidencias en sus mensajes.

Partes: --grep = buscar por mensaje; <patrón> = texto o expresión regular.

`git log <desde>..<hasta>` P. 17-18

Muestra commits alcanzables desde «hasta» que no son alcanzables desde «desde».

Partes: .. = operador de rango de revisiones, no «carpeta superior»; ambos extremos suelen ser ramas o identificadores.

`git log <archivo>` P. 18

Intenta consultar el historial relacionado con un archivo específico.

Partes: El argumento es una ruta. Si se necesita distinguirlo de una referencia, la forma inequívoca es git log -- <archivo>.

`git log --oneline --decorate` P. 14

Resume los commits e indica los nombres de ramas o etiquetas que apuntan a ellos.

Partes: --oneline = una línea; --decorate = referencias asociadas.

`git log --graph --decorate --oneline` P. 18

Dibuja un gráfico de texto del historial con ramas/etiquetas y una línea por commit.

Partes: --graph = trazado; --decorate = referencias; --oneline = resumen.

`git log --oneline --decorate --graph --all` P. 15-16

Muestra gráficamente el historial alcanzable desde todas las referencias, no solo la rama activa.

Partes: --oneline = compacto; --decorate = referencias; --graph = gráfico; --all = todas las referencias.

07 Ramas y cambio de rama

`git branch testing` P. 14

Crea una rama local llamada testing sin cambiar a ella.

Partes: branch = ramas; testing = nombre elegido. La rama nace en el commit actual.

`git checkout testing` P. 14

Cambia a la rama testing y actualiza los archivos de trabajo a esa rama.

Partes: checkout = cambiar de referencia/recuperar; testing = rama de destino.

`git checkout master` P. 15

Cambia a la rama local master, si existe.

Partes: master = nombre de una rama, no opción especial de Git.

`git checkout -b nombre-de-la-nueva-rama` P. 24

Crea la rama indicada y cambia inmediatamente a ella.

Partes: -b = crear una rama en la operación de checkout; lo demás es el nombre de la nueva rama.

08 Enviar, descargar y actualizar: remotos

`git push` P. 10, 18

Publica confirmaciones hacia el remoto configurado, si se conoce la rama de destino.

Partes: push = enviar. Solo envía commits realizados, no cambios sin confirmar.

`git push <remoto> <rama>` P. 18

Envía la rama local indicada al remoto indicado.

Partes: <remoto> es normalmente un alias como origin; <rama> es el nombre de rama.

`git push origin main` P. 19

Publica los commits de la rama local main en el remoto origin.

Partes: origin = alias remoto; main = nombre de la rama. Aparece como ejemplo explícito.

`git push origin master` P. 11 · esquema

Publica los commits de la rama local master en el remoto origin.

Partes: master es un nombre alternativo de rama principal. Aparece en el esquema de la unidad.

`git push --all <remoto>` P. 19

Envía todas las ramas locales al repositorio remoto seleccionado.

Partes: --all = todas las ramas locales (no significa «todos los archivos» ni todas las etiquetas).

`git push --force <remoto> <rama>` P. 19

Fuerza la actualización remota aunque requiera reemplazar historia que existe allí.

Partes: --force = forzar: una opción peligrosa para trabajos compartidos.

`git push --no-verify` P. 19

Omite el hook local pre-push antes del envío.

Partes: --no-verify = no comprobar el hook local; no elimina las validaciones del servidor.

`git pull` P. 20

Descarga e intenta integrar los cambios del remoto configurado en la rama actual.

Partes: pull = obtener + integrar. Según la configuración puede usar merge, rebase o solo fast-forward.

`git pull origin main` P. 11

Obtiene de origin la rama main y procura integrarla en la rama local actual.

Partes: origin = remoto; main = referencia remota que se obtiene.

`git pull OPCIONES REPOSITORIO REFSPEC` P. 20

Describe una sintaxis general, no un comando listo para copiar literalmente.

Partes: OPCIONES = modificadores; REPOSITORIO = remoto/URL; REFSPEC = especificación de referencias.

`git pull <remoto> <rama>` P. 20

Descarga e integra los cambios de una rama concreta del remoto indicado.

Partes: <remoto> y <rama> deben sustituirse por nombres reales.

`git fetch` P. 20

Obtiene los cambios y actualiza las referencias remotas, sin integrar por sí solo los cambios en la rama actual.

Partes: fetch = recuperar datos y referencias. No equivale a pull.

09 Fusión y reorganización de historial

`git merge` P. 20

Intenta integrar la rama o ramas ascendentes configuradas, cuando la configuración lo permite.

Partes: merge = fusionar. Sin argumento necesita una referencia adecuada en la configuración.

`git merge <rama>` P. 23

Integra los cambios de la rama indicada en la rama activa.

Partes: <rama> se sustituye por una rama existente; no se cambia automáticamente a ella.

`git merge new-features` P. 23-24

Fusiona los cambios de new-features en la rama actual.

Partes: new-features es un ejemplo de nombre de rama, no una opción.

`git rebase master` P. 24-25

Reaplica los commits exclusivos de la rama actual tomando master como nueva base.

Partes: rebase = cambiar la base; master = rama sobre la que se reponen los commits.

`git rebase master branch` P. 25

Reaplica los commits de la rama branch sobre master, actualizando branch.

Partes: master = nueva base; branch = rama que se reorganiza; son argumentos posicionales.

10 Guardar temporalmente y descartar

`git stash` P. 24

Guarda temporalmente los cambios controlados y deja libre el espacio de trabajo, en condiciones habituales.

Partes: stash = almacenamiento temporal. Sin opciones no guarda los archivos sin seguimiento.

`git stash pop` P. 24

Reaplica el último conjunto guardado en el stash y elimina su entrada si la aplicación tiene éxito.

Partes: pop = recuperar y extraer del stash. Puede ocasionar conflictos.

`git reset` P. 21

Sin parámetros adicionales, restablece el área de preparación al estado de HEAD y conserva el contenido de trabajo.

Partes: reset = restablecer. No deshace automáticamente el último commit al usarse sin argumentos.

`git reset --hard` P. 24

Restablece área de preparación y archivos controlados al commit actual, descartando sus cambios sin confirmar.

Partes: --hard = modo de restablecimiento destructivo para cambios locales controlados.

11 Órdenes ajenas a Git que también aparecen

`cd <directorio>` P. 18

Se mueve a una carpeta en la terminal; aparece tras clonar un repositorio.

Partes: cd = change directory; el directorio se sustituye por el nombre de carpeta real.

`vim test.rb` P. 15

Abre el archivo test.rb con el editor Vim para modificarlo.

Partes: vim es un editor de terminal; test.rb = fichero que se edita.

`rcs` P. 4

Nombre de una herramienta/comando del sistema histórico de control de revisiones RCS.

Partes: Se menciona como antecedente; la unidad no enseña su sintaxis ni opciones.

`svn add` P. 22

Indica a Subversion que añada una ruta al control de versiones.

Partes: svn pertenece a Subversion y NO es un comando de Git; no se da un argumento concreto en la unidad.

`javac` P. 28

Compilador Java mencionado al comparar la compilación manual con la facilitada por un IDE.

Partes: Se menciona sin parámetros; no se detalla una invocación completa.

05 Detalles dentro de las órdenes interactivas

Qué contestar al ejecutar git add -p

Tecla	Acción	Uso práctico
y	Yes / sí	Preparar el fragmento que Git muestra.
n	No	Dejar ese fragmento fuera de la preparación.
s	Split / dividir	Dividir el bloque de cambios en fragmentos más pequeños si se puede.
e	Edit / editar	Editar manualmente el parche seleccionado.
q	Quit / salir	Salir del modo interactivo sin seguir eligiendo fragmentos.

Qué significan los indicadores de conflicto

El ejemplo de conflicto en la **página 25** muestra marcadores que Git inserta dentro del archivo afectado. Son texto del archivo mientras dura el conflicto; **no son instrucciones de terminal**.

```
<<<<<<< HEAD
Address: 808 South Street
=======
Address: 505 North Street
>>>>>>> updated_address
```

`<<<<<<< HEAD` indica el inicio de la alternativa de la versión actual; **`=======`** separa las dos alternativas; **`>>>>>>> updated_address`** cierra el bloque y nombra la versión que se está integrando.

Otros elementos mencionados

El carácter # en `git stash # agregarlos al stash` comienza una aclaración del ejemplo y no pertenece al comando. En `git fetch + git merge`, el signo **+** es una forma de explicar la composición habitual de pull, no una orden ejecutable.

En la consulta de historial, **`Espacio`** permite avanzar por páginas y **`q`** cerrar el visor cuando se utiliza un paginador. Las teclas son controles interactivos, no opciones de Git.

06 Erratas e imprecisiones del material original

Esta sección diferencia las frases de los apuntes de la interpretación técnica correcta. Su objetivo es evitar que se copien errores de sintaxis o se aprendan comportamientos incorrectos.

01 · Errata de opción · pág. 23

El apunte escribe git commit –amend con un guion tipográfico, mientras la forma correcta es git commit --amend. --amed tampoco es una opción válida.

02 · Interpretación de -n · pág. 17

La descripción de git log -n <límite> atribuye el formato de una línea al comando. En realidad -n limita la cantidad de commits; el formato resumido corresponde a --oneline.

03 · Rama predeterminada · pág. 11, 13

Se afirma que Git crea master de manera predeterminada, pero el nombre depende de la configuración de Git y del repositorio. La unidad también muestra main.

04 · Requisito para init · pág. 9

Se pide comenzar necesariamente con una carpeta vacía para git init. Técnicamente también se puede inicializar Git dentro de una carpeta con archivos existentes.

05 · Diferencia entre . y * · pág. 9, 24

git add . actúa bajo el directorio actual; git add * depende de la expansión del comodín de la terminal y puede omitir archivos ocultos. No son equivalentes.

06 · Confusión sobre commit -am · pág. 22-23

La opción -a NO incorpora archivos nuevos sin seguimiento aunque se combine con -m. Para archivos nuevos hace falta prepararlos.

07 · Merge y commits · pág. 23, 27

Se presenta git merge como una operación que siempre crea un commit. Puede realizar una fusión de avance rápido (fast-forward) sin commit adicional.

08 · Merge y archivos modificados · pág. 23

Se afirma que cualquier cambio local sin commit impide toda fusión. Git puede permitirla si no pone en peligro las modificaciones locales.

09 · Reset sin argumentos · pág. 21

Se presenta git reset de forma general como herramienta para deshacer una confirmación. Ejecutado sin parámetros restablece la preparación, pero no retrocede por sí mismo el historial.

10 · Reset --hard · pág. 24

No «remueve todos los cambios pendientes» sin excepción: descarta cambios de archivos controlados, mientras los archivos nuevos sin seguimiento no se eliminan normalmente.

11 · Push forzado · pág. 19

No debe presentarse como manera ordinaria de «ignorar» cambios de GitHub: --force puede sustituir historial remoto y hacer perder commits de compañeros.

12 · Rebase y colaboración · pág. 24-27

Rebase reescribe confirmaciones de la rama reorganizada. Lo importante es evitar reescribir historia publicada que otros compañeros ya utilizan.

13 · Pull y configuración · pág. 20

La unidad equipara siempre pull a fetch + merge. Es la forma habitual cuando Git está configurado para fusionar, pero existen modos rebase o fast-forward.

14 · Historia de archivo · pág. 18

La sintaxis git log <archivo> puede ser ambigua si coincide con una referencia. La forma más clara para una ruta es git log -- <archivo> (aclaración externa al enunciado).

15 · Ejemplo con comillas corruptas · pág. 18

En la página 18 el ejemplo de git commit -m presenta comillas de apertura/cierre y texto malformados. La forma utilizable es git commit -m "mensaje".

07 Secuencias de trabajo para entender el conjunto

Estos ejemplos utilizan **solo los comandos ya recogidos** (con nombres de archivos o ramas ilustrativos). Sirven para relacionar las opciones y los conceptos de la unidad.

A. Inicio de un proyecto nuevo

Orden (en secuencia)	Por qué se usa
`git init`	Crea el repositorio local.
`git add .`	Prepara los cambios de la carpeta de trabajo.
`git commit -m "Primera versión"`	Guarda la primera versión en el historial local.
`git remote add origin <URL>`	Vincula el repositorio con un remoto, sustituyendo <URL> por la dirección real.
`git push origin main`	Publica main si esa rama existe y su configuración admite el envío.

B. Trabajar en una rama separada

Orden (en secuencia)	Por qué se usa
`git checkout -b nueva-funcionalidad`	Crea y selecciona una rama.
`git add .`	Prepara los cambios.
`git commit -m "Termino funcionalidad"`	Confirma los cambios en la rama de trabajo.
`git checkout master`	Vuelve a master, si esa rama existe.
`git merge nueva-funcionalidad`	Integra la nueva funcionalidad en la rama activa.

C. Comprobar antes de publicar

Orden (en secuencia)	Por qué se usa
`git status -s -b`	Comprueba rama y estado resumido.
`git log --oneline --decorate --graph --all`	Inspecciona el historial gráfico.
`git push origin main`	Envía commits al remoto si estás trabajando con main.

D. Consultar y recuperar cambios de compañeros

Orden (en secuencia)	Por qué se usa
`git fetch`	Descarga referencias remotas sin incorporarlas a tu rama.
`git pull origin main`	Obtiene e intenta integrar los cambios de main.

Recordatorio final: `git add` prepara; `git commit` confirma localmente; `git push` publica; `git fetch` descarga referencias; `git pull` descarga e integra. `git reset --hard` y `git push --force` requieren especial precaución.

Fuente principal: *UT 02 - Apuntes (1).pdf*, 30 páginas, unidad «Control de versiones». El contenido técnico aclaratorio se separa de las erratas del documento en el apartado 06. La paginación indicada en fichas y diccionarios se refiere siempre al original.